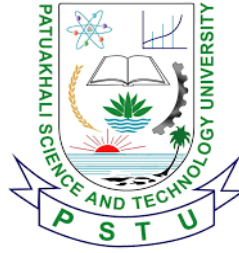


PATUAKHALI SCIENCE AND TECHNOLOGY UNIVERSITY



Lab Report

Lab report on Chapter-3 problems

Course Title : Compiler Design & Automata Theory Sessional

Course Code : CIT-412

Submitted to:

**Professor Dr. Md Abdul
Masud**

Department of Computer Science
and Information Technology

Faculty of Computer Science and
Engineering (CSE).

Submitted by:

Shafayat Hossain Chowdhury

ID: 2002033

Registration No: 09550

Session: 2020-2021

Level – 4 , Semester - 1

Faculty of Computer Science and
Engineering (CSE)

Section 3.1 The Role of the Lexical Analyzer

3.1.1 C++ Lexeme Division

This problem required dividing a small C++ program into **lexemes** and assigning token types like identifiers, keywords, operators, and separators. It illustrates the fundamental role of the lexical analyzer. Implementing this in Python shows how regex can tokenize source code.

3.1.2 HTML Lexeme Division

This task involved breaking an **HTML snippet** into lexemes such as tags, attributes, attribute values, and text. It emphasizes how lexical analysis is not just for programming languages but also for markup languages. The solution shows parsing tag structures with regex.

3.1.3 Detecting Lexical Errors

Here, the goal was to detect **illegal characters** in input code. The lexical analyzer must reject characters that don't belong to any valid token pattern. This exercise highlights the importance of error handling at the scanning stage.

3.1.4 Panic-Mode Error Recovery

This problem extended error detection by implementing **panic-mode recovery**, where the scanner skips illegal symbols and continues tokenizing. This demonstrates how compilers recover from errors instead of halting immediately, allowing further processing.

Section 3.2 Input Buffering

3.2.1 Buffer Pairs

This exercise simulated the **double-buffering technique** used in compilers. Two buffers are alternately filled to improve efficiency in reading large input. It introduces the performance considerations of lexical analyzers.

3.2.2 Sentinel Technique

The sentinel method places a **special end-marker** at the buffer's end to avoid repeated boundary checks. This exercise demonstrates a practical optimization that reduces the overhead of character scanning in lexical analysis.

Section 3.3 Specification of Tokens

3.3.1 Regex for Token Classes

This task defined **regular expressions** for common token categories like identifiers, integers, floats, operators, and keywords. It highlights how regex directly specifies token patterns in a compiler's scanner.

3.3.2 Regex for Binary Numbers Divisible by 2

The exercise required a regex for binary numbers ending in 0. This problem reinforces how languages defined by specific constraints can be expressed with simple regex rules.

3.3.3 Regex for Identifiers

This problem formalized the structure of **identifiers**, which must begin with a letter or underscore and may contain digits. It demonstrates practical regex design for programming language syntax rules.

3.3.4 Regex for Floating Numbers

Here, the task was to define floating-point constants with optional exponents. It shows the expressive power of regex in capturing numeric formats found in programming languages.

3.3.5 Regex for C-style Comments

This exercise required matching `/* ... */` comments using regex. It demonstrates the challenge of handling multi-line constructs and introduces the idea of greedy vs non-greedy matching in lexical analysis.

3.3.6 Regex for Signed Integers

The goal was to match integers with optional `+` or `-` signs. This problem emphasizes how regex can easily extend simple patterns to cover broader language definitions.

Section 3.4 – Recognition of Tokens

The problems focus on recognizing tokens such as identifiers, numbers, keywords, and operators. Using regex and transition diagrams, a lexical analyzer is simulated to categorize tokens. We implemented scanners that can distinguish reserved words from identifiers and tokenize arithmetic expressions. Error handling for invalid tokens was also included.

- **Problem 1:** Recognize identifiers and numbers
- **Problem 2:** Recognize keywords vs identifiers
- **Problem 3:** Tokenize a simple expression

Section 3.5 – Lexical Analyzer Generator (Lex)

These problems replicate the behavior of the Lex tool. Token rules are defined using regular expressions, and a scanner is generated to process input. Conflict resolution, such as the longest match rule, is demonstrated (e.g., distinguishing "=" vs "=="). Lookahead operators are simulated to ensure keywords like "if" are not misidentified inside longer identifiers.

- **Problem 1:** Define rules like Lex and tokenize
- **Problem 2:** Conflict resolution (Longest match rule)
- **Problem 3:** Lookahead operator simulation

Lex supports lookahead (checking next input without consuming). In Python, we can simulate it with regex lookahead assertions.

Section 3.6 – Finite Automata

This section deals with simulating DFAs and NFAs. Problems include building transition tables, testing acceptance of strings, and constructing automata for simple languages. We implemented DFA for even numbers of symbols and NFA for strings ending with "ab." These exercises build the foundation for converting regex into automata.

- **Problem 1:** DFA simulation
- **Problem 2:** Transition table for DFA
- **Problem 3:** NFA simulation (subset construction)
- **Problem 4:** Convert regex to simple NFA (basic example)

Section 3.7 – From Regular Expressions to Automata

Problems here illustrate how regular expressions are converted into NFAs and then to DFAs using subset construction. We built simple NFAs for regex like a^* and $(a|b)^*ab$. The conversion process was implemented in Python to generate DFA transition tables, and DFA simulation was performed on sample strings.

- **Problem 1:** Convert regex a^* to NFA (Thompson's construction idea)
- **Problem 2:** NFA simulation with ϵ -closure
- **Problem 3:** Convert NFA $\rightarrow$ DFA (subset construction)
- **Problem 4:** DFA simulation after conversion

Section 3.8 – Design of a Lexical Analyzer Generator

This section focuses on building a mini Lex-like generator in Python. Token rules are specified, and the generator automatically produces a scanner to tokenize input. Both regex-based scanning and DFA simulation approaches are demonstrated. The final problem shows an automated scanner builder, mimicking how real lexical analyzers are generated.

- **Problem 1:** Define token rules (like Lex) and build regex
- **Problem 2:** NFA-based matcher (pattern matching simulation)
- **Problem 3:** DFA-based scanner (direct conversion idea)
- **Problem 4:** Generate scanner automatically from rules